

This is unreleased documentation for Serverpod **Next** version.

For up-to-date documentation, see the [latest version](#) (4.0.0).

3..25.2

THE BASICS

 Copy as Markdown

Legacy authentication basics

INFO

This page documents the legacy `serverpod_auth` module. To move an existing app to the current authentication framework, see [Migrate from legacy auth](#).

Serverpod automatically checks if the user is logged in and if the user has the right privileges to access the endpoint. When using the `serverpod_auth` module you will not have to worry about keeping track of tokens, refreshing them or, even including them in requests as this all happens automatically under the hood.

The `Session` object provides information about the current user. A unique `userIdentifier` identifies a user. You should use this id whenever you are referring to a user. Access the id of a signed-in user through the `authenticated` getter of the `Session` object. Since the default implementation of `serverpod_auth` uses numeric IDs for the users, there is a convenience getter `userId` on the `AuthenticationInfo`, which returns the integer value.

```
Future<void> myMethod(Session session) async {
  final authenticationInfo = session.authenticated;
  final userIdentifier = authenticationInfo?.userIdentifier;
  final userId = authenticationInfo?.userId;
  ...
}
```

You can also use the `Session` object to check if a user is authenticated:

```
Future<void> myMethod(Session session) async {
  var isSignedIn = session.isUserSignedIn;
  ...
}
```



Requiring authentication on endpoints

It is common to want to restrict access to an endpoint to users that have signed in. You can do this by overriding the `requireLogin` property of the `Endpoint` class.

```
class MyEndpoint extends Endpoint {  
  @override  
  bool get requireLogin => true;  
  
  Future<void> myMethod(Session session) async {  
    ...  
  }  
  ...  
}
```

Explicitly allowing unauthenticated access

In some cases, you may want to explicitly allow certain endpoints or methods to be accessed without authentication. Serverpod provides the `@unauthenticatedClientCall` annotation for this purpose.

When an endpoint or method is annotated with `@unauthenticatedClientCall`:

- No authentication will be added to the header on the client when calling it.
- The server will receive calls as if there is no user signed in.

INFO

Under the hood, the `@unauthenticatedClientCall` annotation makes the client omit authentication headers for calls to the annotated endpoint or method. On the server side, it ensures that the session is treated as unauthenticated for those calls, regardless of any existing authentication state.

You can use this annotation in two ways:

1. On the entire endpoint class to make all methods unauthenticated:

```
@unauthenticatedClientCall  
class UnauthenticatedEndpoint extends Endpoint {  
  Future<bool> someMethod(Session session) async {  
    return session.isUserSignedIn; // Will always return false  
  }  
  
  Stream<bool> someStream(Session session) async* {  
    yield session.isUserSignedIn; // Will always return false  
  }  
}
```

2. On specific methods to make only those methods unauthenticated:

```
class PartiallyUnauthenticatedEndpoint extends Endpoint {
  @unauthenticatedClientCall
  Future<bool> publicMethod(Session session) async {
    return session.isUserSignedIn; // Will always return false
  }

  Future<bool> authenticatedMethod(Session session) async {
    return session.isUserSignedIn;
  }
}
```

This is particularly useful for endpoints that must not receive authentication, such as JWT refresh endpoints.

⚠ WARNING

Using `@unauthenticatedClientCall` on an endpoint or method that also has `requireLogin` set to true will lead to a conflict. Since the client will suppress sending authentication information, but the server will expect it, calls to such endpoints or methods will always fail with an authentication error.

Authorization on endpoints

Serverpod also supports scopes for restricting access. One or more scopes can be associated with a user. For instance, this can be used to give admin access to a specific user. To restrict access for an endpoint, override the `requiredScopes` property. Note that setting `requiredScopes` implicitly sets `requireLogin` to true.

```
class MyEndpoint extends Endpoint {
  @override
  bool get requireLogin => true;

  @override
  Set<Scope> get requiredScopes => {Scope.admin};

  Future<void> myMethod(Session session) async {
    ...
  }
  ...
}
```

Managing scopes

New users are created without any scopes. To update a user's scopes, use the `Users` class's `updateUserScopes` method (requires the `serverpod_auth_server` package). This method replaces all previously stored scopes.

```
await Users.updateUserScopes(session, userId, {Scope.admin});
```

Custom scopes

You may need more granular access control for specific endpoints. To create custom scopes, extend the `Scope` class, as shown below:

```
class CustomScope extends Scope {  
  const CustomScope(String name) : super(name);  
  
  static const userRead = CustomScope('userRead');  
  static const userWrite = CustomScope('userWrite');  
}
```

Then use the custom scopes like this:

```
class MyEndpoint extends Endpoint {  
  @override  
  bool get requireLogin ⇒ true;  
  
  @override  
  Set<Scope> get requiredScopes ⇒ {CustomScope.userRead, CustomScope.userWrite};  
  
  Future<void> myMethod(Session session) async {  
    ...  
  }  
  ...  
}
```

⚠ CAUTION

Keep in mind that a scope is merely an arbitrary string and can be written in any format you prefer. However, it's crucial to use unique strings for each scope, as duplicated scope strings may lead to unintentional data exposure.

User authentication

The `StatusEndpoint` class includes methods for handling user sign-outs, whether from a single device or all devices.

ℹ INFO

In addition to the `StatusEndpoint` methods, Serverpod provides more comprehensive tools for managing user authentication and sign-out processes across multiple devices.

For more detailed information on managing and revoking authentication keys, please refer to the [Revoking authentication keys](#) section.

Sign out device

To sign out a single device:

```
await client.modules.auth.status.signOutDevice();
```

This status endpoint method obtains the authentication key from session's authentication information, then revokes that key.

Sign out all devices

To sign the user out across all devices:

```
await client.modules.auth.status.signOutAllDevices();
```

This status endpoint retrieves the user ID from session's authentication information, then revokes all authentication keys related to that user.

INFO

The `signOut` status endpoint was removed in the 3.0 release of the module. Use `signOutDevice` or `signOutAllDevices` instead.